

Jad Hdeife, Ahmad El Hariri
CMPS 261 License Plate Project Report

Introduction

License plate detection is a very well-known computer vision task usually used in traffic monitoring, parking systems, and toll collection. The goal of our project is to locate, using models that are already trained on detecting objects, the license plate in an image by predicting a bounding box around it. The task is easy for clear plates, and becomes much harder when the plates in the dataset are small, tilted, partially hidden, or placed in cluttered backgrounds.

Our objective of this project was to study license plate detection on a dataset of 433 car images and to compare how three different object detection architectures we chose perform on the same task: **YOLOv8n**, **Faster R-CNN (ResNet-50 FPN v2)**, and **RetinaNet (ResNet-50 FPN v2)**. We chose these models because they represent different detection strategies, which will help us explore how different architectures choice affects F1-score, mAP50, mAP50-95, and confidence scores. At the end of the project, we also did a bootstrap analysis to check whether the top two models are statistically separable on the small test set and to choose the better model.

Dataset Exploration and Preprocessing

We used the **Car Plate Detection** dataset from Kaggle, which contains **433 car images** with Pascal VOC XML annotations. During our first inspection, we saw a single class named **licence** and a total of **471 annotated plates**, meaning some images probably contain more than one plate. In order to use YOLO models, YOLO format is needed, so XML files were converted into YOLO normalized text labels (class ID, x-center, y-center, width, height), mapping the class to ID 0. After we converted the files, all 433 files were converted successfully with no empty labels.

To train the models, we split the dataset with a fixed seed into **303 training / 65 validation / 65 test** images, and we organized them in the standard YOLO folder structure (images/ and labels/ subfolders per split) with a license_plate.yaml file describing the dataset. We used the same split for all the three models so that differences in results come from the models, not the data.

Methodology

We trained all the models from pretrained weights on the same split. We set 25 epochs at 640x640 and we used the ultralytics library to train the YOLOv8n model. For **Faster R-CNN**, we trained it for 20 epochs and **RetinaNet** for 25 epochs, both using TorchVision with SGD, momentum, weight decay, a step-based scheduler, and mixed precision when GPU was available (which was always the case since we used the T4 GPU). TorchVision detectors do not use YOLO text labels directly, so we had to convert bounding boxes in memory into (x1, y1, x2, y2) form during data loading.

Our evaluation was done on both validation and test splits using **precision, recall, F1-score, mAP50, and mAP50-95**, with mAP50-95 treated as the main metric for model performance followed by the F1-score. For each TorchVision model, we chose the F1-optimal confidence threshold on the validation set and then reused on the test set, which helped us avoid tuning on test. We did in the end the same thing for the YOLO model during the bootstrap comparison, so all three models were eventually compared fairly. Beyond the main metrics, we also used confidence distributions, sample predictions, and training curves in order to analyze model overfitting and generalization. In the end, we used the **paired bootstrap with 500 resamples** of the test set to compute 95% confidence intervals and p-values to choose the best model among the top 2 models.

Selected Models and Justification

YOLOv8n is a lightweight one-stage detector (~3M parameters) that predicts boxes and class scores in a single forward pass. We chose it as the **baseline** model because it is fast, simple, and it is a strong reference point for comparison with the other models. **Faster R-CNN with ResNet-50 FPN v2** (~43.7M parameters) is a two-stage detector that first proposes regions through an RPN and then classifies and refines them. We included it to compare a heavy, region-based detector against the lightweight YOLO. **RetinaNet with ResNet-50 FPN v2** (~38.2M parameters) is a one-stage detector trained with

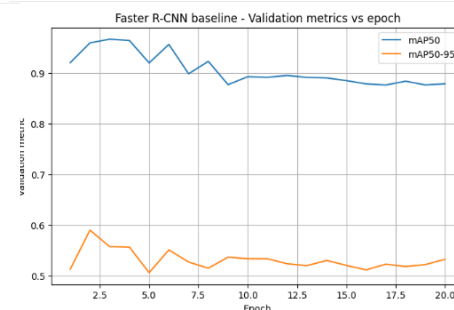
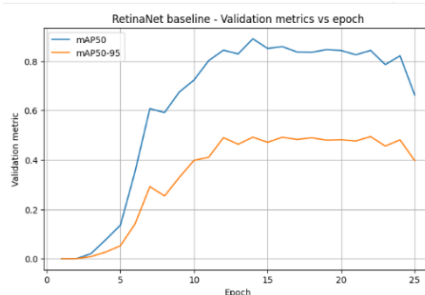
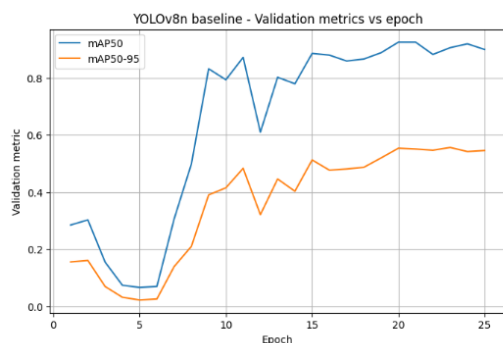
focal loss to handle the imbalance between easy background and hard foreground anchors. We included it as a middle ground: it is a one-stage like YOLO but it also is much heavier, with a different training strategy. These three models cover a one-stage light model, a two-stage heavy model, and a one-stage heavy model with focal loss, which makes the comparison informative.

Evaluation Metrics and Results

Precision and recall describe false positives and false negatives, F1-score balances both, mAP50 is a forgiving localization metric at IoU 0.50, and mAP50-95 averages across IoU thresholds from 0.50 to 0.95 (mAP is mean average precision). We used mAP50-95 as the main comparison metric because it is the metric that reflects strict localization quality. The combined test and validation results are shown below.

Model	Split	Thresh.	Precision	Recall	F1	mAP50	mAP50-95
YOLOv8n	val	-	0.9383	0.8566	0.8956	0.9012	0.5583
YOLOv8n	test	-	0.9446	0.8857	0.9142	0.9505	0.5605
Faster R-CNN	val	0.70	0.9041	0.9296	0.9167	0.9597	0.5900
Faster R-CNN	test	0.70	0.8684	0.9429	0.9041	0.9611	0.5452
RetinaNet	val	0.45	0.9016	0.7746	0.8333	0.8426	0.4940
RetinaNet	test	0.45	0.8429	0.8429	0.8429	0.8589	0.4878

Table 1. Validation and test results for the three models. Thresh. is the F1-optimal confidence threshold picked on validation and reused on test. YOLO used the ultralytics defaults for this initial evaluation; it is re-evaluated at its own F1-optimal threshold in the bootstrap section.



YOLOv8n

YOLO gave the strongest point-estimate results, with the highest test **precision (0.9446)**, **F1 (0.9142)**, and **mAP50-95 (0.5605)**. Its training curve was the healthiest of the three: training loss decreased steadily, validation mAP50-95 improved throughout training, the best epoch was **23**, and the late decline was very small. This looks like a mild plateau rather than overfitting.

Faster R-CNN

Faster R-CNN achieved the highest **recall (0.9429)** and highest **mAP50 (0.9611)**, meaning it was very effective at finding the plate region. However, its training curve showed clear early overfitting: the best validation mAP50-95 (0.5900) was reached

at **epoch 2** and declined afterward while training loss kept dropping. The best checkpoint used for evaluation was therefore the one saved at epoch 2, not the final state.

RetinaNet

RetinaNet was the weakest on every headline metric. Its training curve tells a specific story: validation mAP50-95 improved until about **epoch 13**, then stayed essentially flat around 0.48 for about ten epochs, then dropped sharply to **0.3967** in the final three epochs. The recorded best was epoch 22 at 0.4940, but this peak is only marginally above the plateau and is likely a lucky oscillation rather than real improvement, which means the saved “best” weights are already sitting on the edge of an overfitting region.

Confidence and overfitting

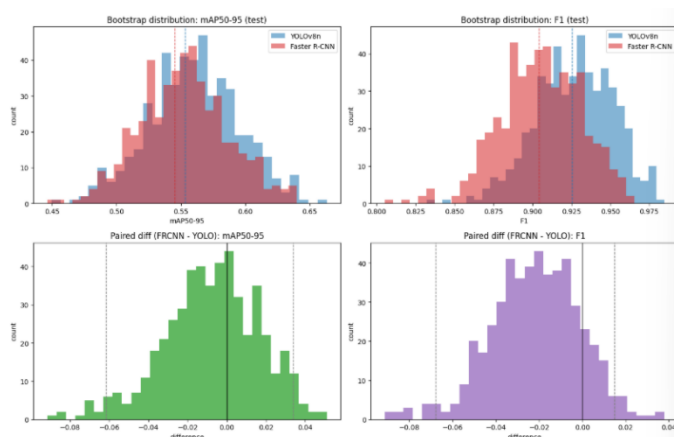
The three models also differ sharply in confidence behavior. On the 65 test images, YOLO produced **77 boxes** with mean confidence **0.6950** and median 0.7400. Faster R-CNN produced **194 boxes** with mean **0.4741** and median 0.3033. RetinaNet produced **1405 boxes** with mean **0.1527** and median 0.0969. This is consistent with how each architecture works: YOLO is selective and decisive, Faster R-CNN proposes more uncertain candidates, and RetinaNet as a dense one-stage detector with focal loss fires at many anchors and keeps low-confidence proposals. The overfitting pattern also differs: YOLO shows a mild plateau, Faster R-CNN overfits very early (epoch 2), and RetinaNet plateaus by epoch 13 and collapses at the end. YOLO is clearly the most stable of the three on this small dataset.

Bootstrap Confidence Intervals and Statistical Comparison

The point-estimate gap between YOLO and Faster R-CNN is small and the test set is only 65 images, so we used a **paired bootstrap** with 500 resamples to check whether that gap is statistically meaningful. For fairness, we re-evaluated YOLO at its own F1-optimal validation threshold (same procedure as Faster R-CNN). We excluded RetinaNet from this step because its point estimates are already clearly below the other two. The bootstrap computes mAP50-95 and F1 for both models on each resampled set of 65 test images, giving a distribution of values and a distribution of paired differences (Faster R-CNN minus YOLO).

Metric	Model	Point est.	95% CI
mAP50-95	YOLOv8n	0.5532	[0.4880, 0.6275]
mAP50-95	Faster R-CNN	0.5452	[0.4877, 0.6215]
F1-score	YOLOv8n	0.9254	[0.8769, 0.9702]
F1-score	Faster R-CNN	0.9041	[0.8571, 0.9517]

Metric	Mean diff (FRCNN - YOLO)	95% CI of diff	p-value
mAP50-95	-0.0078	[-0.0616, +0.0339]	0.7425
F1-score	-0.0218	[-0.0678, +0.0150]	0.2914



On mAP50-95, the two models are essentially tied. The paired difference is centered at -0.0078 with a CI that clearly crosses zero and a p-value of 0.7425, so there is no evidence of a real gap. On F1, YOLO leads in point estimates (0.9254 vs 0.9041) and the paired difference leans in its favor (mean -0.0218), but the 95% CI [-0.0678, +0.0150] still crosses zero with a p-value of 0.2914. So the trend favors YOLO, but the gap is not statistically significant on only 65 test images. Neither metric gives Faster R-CNN a real advantage.

Conclusions

Based on point estimates, **YOLOv8n** was the best model that we used in this project, it had the strongest test precision, F1, and mAP50-95, a healthy training curve, and it had much higher confidence per detection. Faster R-CNN was strong on coarse localization and recall but it overfit very early, and RetinaNet was the weakest overall with a flat plateau followed by a late collapse in its validation curve. The bootstrap analysis showed that YOLO and Faster R-CNN are statistically tied on mAP50-95 and that YOLO's F1 advantage is a non-significant trend, so the gap between them is inside the noise of a 65-image test set. Once the headline metrics are tied, our decision falls on the other evidence, and YOLO wins on every other axis: it is the lightest model, produces fewer and more confident detections, has the most stable training behavior, and is the most practical to deploy. For this dataset and task, **YOLOv8n is the recommended model**. Our natural next steps would be testing a larger image size, adding regularization or earlier stopping for the heavier models (RetinaNet in particular would benefit from stopping around epoch 13), augmenting the dataset, and adding an OCR stage after detection.